CarPark Problem

Simple Replicator with Pure Strategies

In[ ]:=
```
parameters = {d1 → 20, d2 → 2, α → 1, β → 5};
```

In[ ]:=
```
payoff[x_, y_] :=
 (* x going to carpark A, y going to carpark B *)
 (* returns tuple with payoff per individual (carpark A, carpark B)  *)
 {1/(d1 + α x), 1/(d2 + β y)} /. parameters
```

We are playing an *n-Player* game. Make sure that the population is much larger than *n*.

In[ ]:=
```
populationSize = 100;
```

In[ ]:=
```
gameSize = 10;
```

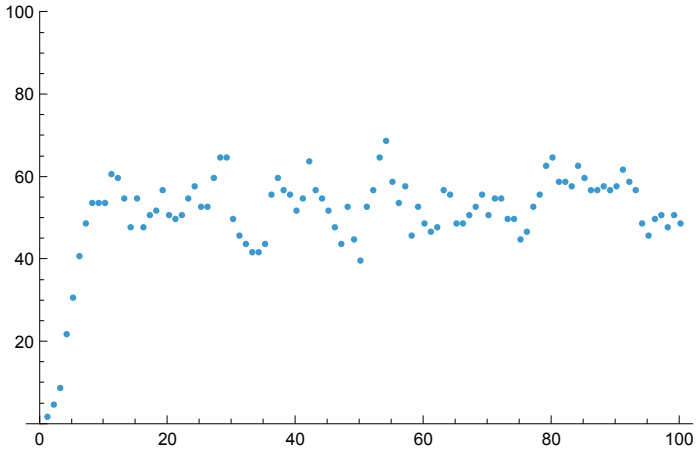
In[ ]:=
```
payoffSingleGame[indexList_] :=
 Module[{traitValues = population[[indexList]], visitorsA, visitorsB},
  {visitorsA, visitorsB} = Table[Count[traitValues, i], {i, 1, 2}];
  payoffs = payoff[visitorsA, visitorsB];
  Map[payoffs[[#]] &, traitValues]
 ]
```

In[ ]:=
```
oneRound[] :=
 Module[{allGames, traitsInGameOrder, payoffsInGameOrder},
  allGames = Partition[RandomSample[Range[populationSize]], gameSize];
  traitsInGameOrder = population[[allGames // Flatten]];
  payoffsInGameOrder = Map[payoffSingleGame, allGames] // Flatten;
  population =
   RandomChoice[payoffsInGameOrder → traitsInGameOrder, populationSize]
 ]
```

In[•]:=
```
play[tEnd_, plotInterval_] :=
 Module[{},
  population = RandomChoice[{0.01, 0.99} → {1, 2}, populationSize];
  For[t = 1, t ≤ tEnd, t++,
   If[Mod[t, plotInterval] == 0, Sow[{{t, Count[population, 1]}}]];
   oneRound[]
  ]
 ]
```

In[•]:=
```
plotEvolution[tEnd_, plotInterval_] :=
 (data = Flatten[Reap[play[tEnd, plotInterval]][[2]], 2];
  Print[ListPlot[data, PlotRange → {0, populationSize}]]
 )
```

In[•]:=
```
Timing[plotEvolution[100, 1]]
```



Out[•]=
```
{0.247875, Null}
```

Play with gamesizes 10,20. What do you observe for the payoff at both carparks in steady-state. Does this agree with your expectations? Explain the reasons for your reasoning.

The fractions do not actually steady into a steady state but fluctuate around it instead. You need large population sizes (n>1,000) and should use sufficiently long simulation horizons (t=1,000) to see easily interpretable reasonable results.

Payoff should be equal for both carparks. The exact steady-state solution for our current games size (10) is:

In[•]:=
```
Solve[x + y == gameSize && (1/(d1 + α x) == 1/(d2 + β y)) /. parameters, {x, y}]
```

Out[•]=
$$\left\{\left\{x \to \frac{16}{3},\ y \to \frac{14}{3}\right\}\right\}$$

In[•]:=
```
% // N
```

Out[•]=
```
{{x → 5.33333, y → 4.66667}}
```

but this can't be achieved since we are dealing with discrete (integer) populations. So no game can

ever settle on exactly equal payoff . For example, for game size 10, we have:

In[•]:=
```
payoff[5, 5]
```
Out[•]=
$$\left\{\frac{1}{25}, \frac{1}{27}\right\}$$

In[•]:=
```
payoff[6, 4]
```
Out[•]=
$$\left\{\frac{1}{26}, \frac{1}{22}\right\}$$

In[•]:=
```
payoff[4, 6]
```
Out[•]=
$$\left\{\frac{1}{24}, \frac{1}{32}\right\}$$

Analytic Solution of the Replicator Equation

Since the analytic RD plays out in an infinite population, whereas we have finite game sizes in the simulation above, we first need to rescale the parameters.

d1 and d2 are absolute constants and don't need to be rescaled. They don't relate to the number of cars. However, α and β do and thus need to be rescaled. Above, the game size is 10 and α is the delay per car. A singe car is 1/10-th of the game size. Thus, the parameters need to be rescaled so that they have the same effect for 1/10-th of the population size, i.e. for an change in x of 0.1.

$\overline{\alpha} = 10\,\alpha;\ \overline{\beta} = 10\,\beta$

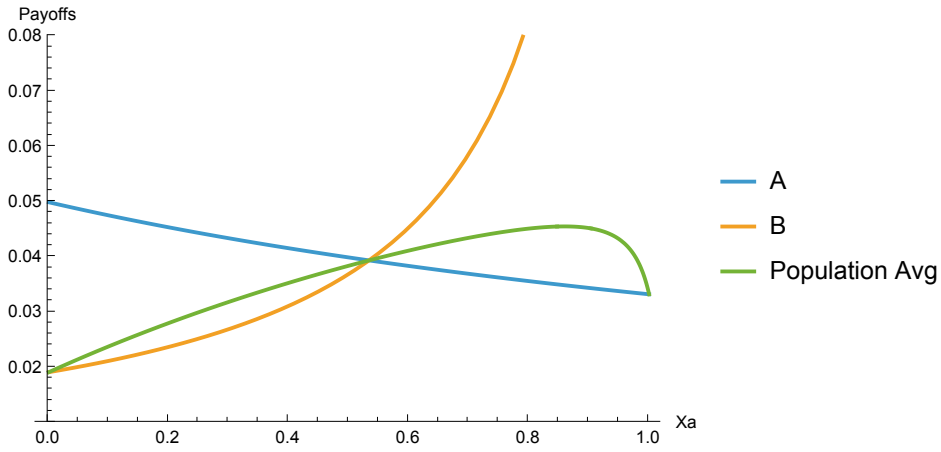
In[•]:=
```
parameters = {d1 → 20, d2 → 2, α̅ → 10, β̅ → 50};
```

In[•]:=
```
payoff[x_, y_] :=
 (* x going to carpark A, y going to carpark B *)
 (* returns tuple with payoff per individual (carpark A, carpark B)  *)
 {1/(d1 + α̅ x), 1/(d2 + β̅ y)} /. parameters
```

In[•]:=
```
avgPayoff[x_] :=
 Module[{payoffs = payoff[x, 1 - x]},
  {x, 1 - x}.payoffs
 ]
```

In[•]:=
```
Plot[
 {payoff[x, 1 - x][[1]], payoff[x, 1 - x][[2]], avgPayoff[x]},
 {x, 0, 1},
 PlotLegends → {"A", "B", "Population Avg"},
 AxesLabel → {"Xa", "Payoffs"},
 PlotRange → {0.01, 0.08}
]
```

Out[•]=



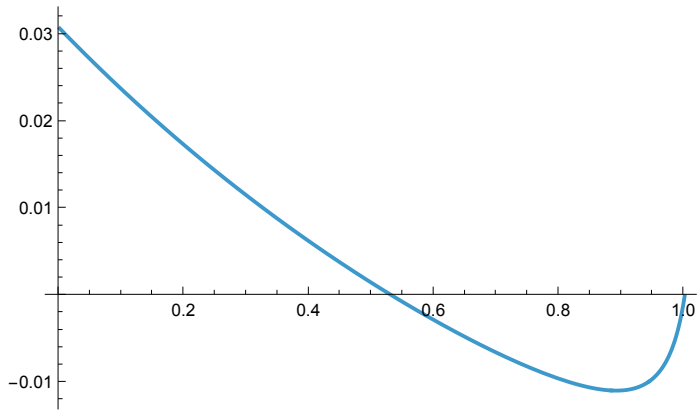
The point where all curves intersect is Nash and the convergence point of the RD. Both payoffs are identical so there is no incentive for any player to deviate from their current strategy. The advantage of either player over the average payoff is likewise zero, so the growth of both strategies is zero and we are in equilibrium.

This is even more clearly seen if we just plot the advantage of one action, say going to A, over the average. The root of this equation is the equilibrium point for this strategy.

In[•]:=
```
advantageA[x_] := payoff[x, 1 - x][[1]] - avgPayoff[x]
```

In[•]:=
```
Plot[advantageA[x], {x, 0, 1}]
```

Out[•]=



In[•]:=
```
Solve[advantageA[x] == 0, x]
```

Out[•]=

$$\left\{\left\{x \to \frac{8}{15}\right\}, \{x \to 1\}\right\}$$

In[•]:=
```
%[[1]] // N
```

Out[•]=
```
{x → 0.533333}
```

We see that the analytic solution and the Monte Carlo simulation agree: in a game of size 10, the equilibrium is when 10*x=80/15 =~= 5.3 cars go to Car Park A.

Analytic Solution of the Replicator Equation - Version 2

above we used the inverse of the delay as the utility. We could instead have calculated with negative utilities. The reason to avoid that is that it would have made the implementation of the Monte Carlo simulation (a little bit) less straight forward.

In the analytic treatment it is entirely straight forward as seen below.

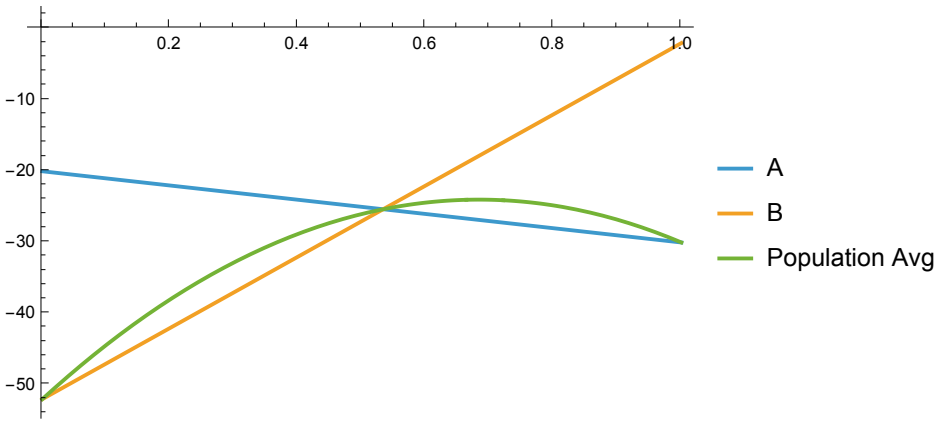
In[•]:=
```
parameters = {d1 → 20, d2 → 2, α → 10, β → 50};
```

In[•]:=
```
payoff[x_, y_] :=
 (* x going to carpark A, y going to carpark B *)
 (* returns tuple with payoff per individual (carpark A, carpark B)  *)
 {-(d1 + α x), -(d2 + β y)} /. parameters
```

In[•]:=
```
avgPayoff[x_] :=
 Module[{payoffs = payoff[x, 1 - x]},
  {x, 1 - x}.payoffs
 ]
```

In[•]:=
```
Plot[
 {payoff[x, 1 - x][[1]], payoff[x, 1 - x][[2]], avgPayoff[x]},
 {x, 0, 1},
 PlotLegends → {"A", "B", "Population Avg"}
]
```

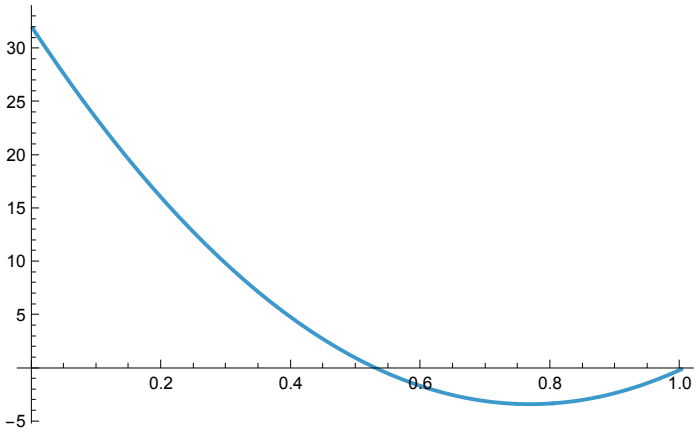
Out[•]=



In[•]:=
```
advantageA[x_] := payoff[x, 1 - x][[1]] - avgPayoff[x]
```

In[•]:=
```
Plot[advantageA[x], {x, 0, 1}]
```

Out[•]=



In[•]:=
```
Solve[advantageA[x] == 0, x]
```

Out[•]=

$$\left\{\left\{x \to \frac{8}{15}\right\}, \{x \to 1\}\right\}$$

Since $\frac{1}{a}$ == $\frac{1}{b}$ implies a==b, we do of course have the same equilibrium.